State Estimation using Kalman Filter (State Estimation, KF, EKF, Damped Harmonic Oscillator, Cart-Pole)

Robotic Systems I

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

Department of Electrical and Computer Engineering
University of Patras

March 23, 2026

Table of Contents

State Estimation in Robotics - What is it and How it Works in Robotics?

In control, automation and robotics, **state estimation** aims to recover the **true internal state** $\boldsymbol{x}$ of a system from its known control inputs $\boldsymbol{u}$ and the noisy measurements $\boldsymbol{z}$ of its true outputs $\boldsymbol{y}$, while accounting for **motion/process noise** $\boldsymbol{w}$ and **observation/measurement noise** $\boldsymbol{v}$. The noise may come from many different sources, like *imperfect incomplete models*, *noisy sensors*, *unknown external disturbances*, etc, so we need a way to get *reliable state estimates* for control and decision making. The key idea is to combine:

1. a **motion/process model** (how the system moves forward / evolves)
2. an **observation/measurement model** (what the sensors observe)

General discrete-time state space model with additive Gaussian noise:

$$\boldsymbol{x}_k = \boldsymbol{f}_d(\boldsymbol{x}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{w}_{k-1}$$

$$\boldsymbol{z}_k = \boldsymbol{h}_d(\boldsymbol{x}_k) + \boldsymbol{v}_k, \quad \boldsymbol{y}_k = \boldsymbol{h}_d(\boldsymbol{x}_k)$$

Common estimation loop (used in Kalman Filter):

1. **Prediction:** use process model to predict next state
2. **Correction:** update estimation using sensor measurements

Discretization & Linearization - Linear Discrete Systems

Let's assume the classical **continuous-time state space model**:

$$\dot{\boldsymbol{x}}(t) = A\boldsymbol{x}(t) + B\boldsymbol{u}(t)$$
$$\boldsymbol{y}(t) = C\boldsymbol{x}(t) + D\boldsymbol{u}(t)$$

We can analytically **discretize this linear system** with **sampling time** dt, assuming **zero-order hold** for the input (meaning it remains constant between successive time steps), as follows:

$$\boldsymbol{x}_{k+1} = A_d\boldsymbol{x}_k + B_d\boldsymbol{u}_k$$
$$\boldsymbol{y}_k = C_d\boldsymbol{x}_k + D_d\boldsymbol{u}_k$$

where:

$$A_d = e^{Adt}, \quad B_d = \left(\int_0^{dt} e^{A\tau}\, d\tau\right) B = A^{-1}(e^{Adt} - \mathcal{I})B, \ \textit{if } \exists A^{-1}$$

$$C_d = C, \quad D_d = D$$

We could also compute discretizations using higher-order holds, like *first-order / piecewise-linear hold*.

Discretization & Linearization - Nonlinear Discrete Systems

For a **nonlinear continuous-time system**:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), \boldsymbol{u}(t))$$

In general, we **cannot compute an analytical discretization**, so we use **numerical integration methods** to approximate:

$$\boldsymbol{x}_{k+1} \approx \boldsymbol{f}_d(\boldsymbol{x}_k, \boldsymbol{u}_k)$$

Explicit Euler (1st order), very simple but not so accurate:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + dt \cdot \boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k)$$

Runge-Kutta 4th order (RK4), more accurate and widely used in practice:

$$\boldsymbol{k_1} = \boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k)$$

$$\boldsymbol{k_2} = \boldsymbol{f}(\boldsymbol{x}_k + \frac{dt}{2} \cdot \boldsymbol{k_1}, \boldsymbol{u}_k)$$

$$\boldsymbol{k_3} = \boldsymbol{f}(\boldsymbol{x}_k + \frac{dt}{2} \cdot \boldsymbol{k_2}, \boldsymbol{u}_k)$$

$$\boldsymbol{k_4} = \boldsymbol{f}(\boldsymbol{x}_k + dt \cdot \boldsymbol{k_3}, \boldsymbol{u}_k)$$

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{dt}{6}(\boldsymbol{k_1} + 2\boldsymbol{k_2} + 2\boldsymbol{k_3} + \boldsymbol{k_4})$$

Discretization & Linearization - Linearization of Nonlinear Discrete Systems

For a **nonlinear discrete-time system** $\boldsymbol{x}_{k+1} \approx \boldsymbol{f}_d(\boldsymbol{x}_k, \boldsymbol{u}_k)$:

We **linearize via Taylor expansion** around $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}})$ and approximate locally as:

$$\boldsymbol{x}_{k+1} \approx \boldsymbol{f}_d(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) + A_d \cdot (\boldsymbol{x}_k - \bar{\boldsymbol{x}}) + B_d \cdot (\boldsymbol{u}_k - \bar{\boldsymbol{u}}), \quad A_d = \frac{\partial \boldsymbol{f}_d}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, B_d = \frac{\partial \boldsymbol{f}_d}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}$$

Euler Integration (1st-order):

$$A_d = \mathcal{I} + dt \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, \quad B_d = dt \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}$$

Runge-Kutta 4th-order (RK4):

$$\frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}} \qquad \frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{u}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}$$

$$\frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_1}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \left(\mathcal{I} + \frac{dt}{2}\frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{x}}\right) \qquad \frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{u}} = \frac{dt}{2}\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_1}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{u}} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_1}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}}$$

$$\frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_2}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \left(\mathcal{I} + \frac{dt}{2}\frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{x}}\right) \qquad \frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{u}} = \frac{dt}{2}\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_2}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{u}} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}}+\frac{dt}{2}\boldsymbol{k_2}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}}$$

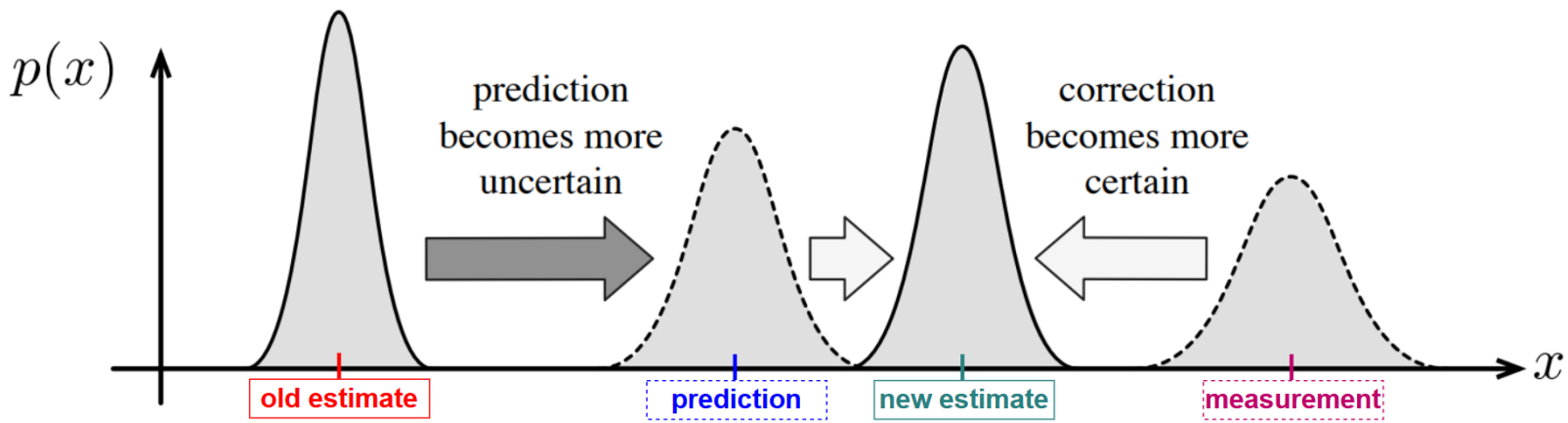
$$\frac{\partial \boldsymbol{k_4}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+dt\boldsymbol{k_3}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \left(\mathcal{I} + dt\frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{x}}\right) \qquad \frac{\partial \boldsymbol{k_4}}{\partial \boldsymbol{u}} = dt\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\Big|_{\bar{\boldsymbol{x}}+dt\boldsymbol{k_3}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}} \frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{u}} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\Big|_{\bar{\boldsymbol{x}}+dt\boldsymbol{k_3}(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}),\bar{\boldsymbol{u}}}$$

$$A_d = \mathcal{I} + \frac{dt}{6}\left(\frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{x}} + 2\frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{x}} + 2\frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{x}} + \frac{\partial \boldsymbol{k_4}}{\partial \boldsymbol{x}}\right), \quad B_d = \frac{dt}{6}\left(\frac{\partial \boldsymbol{k_1}}{\partial \boldsymbol{u}} + 2\frac{\partial \boldsymbol{k_2}}{\partial \boldsymbol{u}} + 2\frac{\partial \boldsymbol{k_3}}{\partial \boldsymbol{u}} + \frac{\partial \boldsymbol{k_4}}{\partial \boldsymbol{u}}\right)$$

Kalman Filter - What is it?

Kalman Filter (KF) was first introduced by Rudolf E. Kálmán, in his 1960 paper *"A New Approach to Linear Filtering and Prediction Problems"*. KF is the **optimal bayes state estimator** for *linear system models* with **additive white Gaussian noise** in both the *motion/process* and the *observation/measurement model*. It works recursively in two stages, alternating between ***Prediction*** and ***Correction*** at each time step:

1. **Prediction**
   The prediction step propagates the old estimate forward in time, using the measurement model and latest input, to arrive at the prediction.
2. **Correction**
   The correction step fuses the prediction with the latest measurement, to arrive at the new estimate, which is done using a normalized product of the two Gaussians.



Prediction and Correction stages of Kalman Filter visualized

Kalman Filter - How it Works in Linear Systems (KF)?

Below is the Kalman Filter algorithm for **linear systems**. We assume ***discrete time-dependent*** state space matrices A, B, C (we removed d subscripts for simplicity). Here, we omit D as it is often zero or neglected in many physical dynamical systems (if it existed, it would only affect the *innovation* term). We assume additive white Gaussian noise $\boldsymbol{w}_k \sim \mathcal{N}(\mathbf{0}, Q_k)$, $\boldsymbol{v}_k \sim \mathcal{N}(\mathbf{0}, R_k)$, where Q_k and R_k are the **process** and **measurement noise covariance matrices**, respectively.

1. **Prediction Stage** (inputs: $\boldsymbol{\mu}_{k-1}$, $\boldsymbol{u}_{k-1}$, Σ_{k-1})

$$\bar{\boldsymbol{\mu}}_k = A_k \boldsymbol{\mu}_{k-1} + B_k \boldsymbol{u}_{k-1} \quad \text{(predicted state estimate)}$$
$$\bar{\Sigma}_k = A_k \Sigma_{k-1} A_k^\top + Q_{k-1} \quad \text{(predicted covariance estimate)}$$

2. **Correction Stage** (outputs: $\boldsymbol{\mu}_k$, Σ_k)

$$K_k = \bar{\Sigma}_k C_k^\top (C_k \bar{\Sigma}_k C_k^\top + R_k)^{-1} \quad \text{(optimal Kalman gain)}$$
$$\boldsymbol{\mu}_k = \bar{\boldsymbol{\mu}}_k + K_k \underbrace{(\boldsymbol{z}_k - C_k \bar{\boldsymbol{\mu}}_k)}_{\textit{innovation}} \quad \text{(corrected state estimate)}^1$$
$$\Sigma_k = (\mathcal{I} - K_k C_k) \bar{\Sigma}_k \quad \text{(corrected covariance estimate)}$$

[1] $\boldsymbol{\mu}_k = (\mathcal{I} - K_k C_k)\bar{\boldsymbol{\mu}}_k + K_k \boldsymbol{z}_k$ (linear interpolation between the predicted state estimate and the measurement)

Kalman Filter - How it Works in Nonlinear Systems (EKF)?

The **Extended Kalman Filter (EKF)** generalizes the Kalman Filter to **nonlinear systems**. We assume a **discrete-time nonlinear system** $\boldsymbol{x}_k = \boldsymbol{f}_d(\boldsymbol{x}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{w}_{k-1}$, $\boldsymbol{z}_k = \boldsymbol{h}_d(\boldsymbol{x}_k) + \boldsymbol{v}_k$, with additive Gaussian noise $\boldsymbol{w}_k \sim \mathcal{N}(\boldsymbol{0}, Q_k)$, $\boldsymbol{v}_k \sim \mathcal{N}(\boldsymbol{0}, R_k)$. To apply the EKF, we **linearize** the system around the current state estimates using the Jacobians: $F_k = \left.\frac{\partial \boldsymbol{f}_d}{\partial \boldsymbol{x}}\right|_{\boldsymbol{\mu}_{k-1}, \boldsymbol{u}_{k-1}}$, $H_k = \left.\frac{\partial \boldsymbol{h}_d}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{\mu}}_k}$ [2].

1. **Prediction Stage** (inputs: $\boldsymbol{\mu}_{k-1}$, $\boldsymbol{u}_{k-1}$, Σ_{k-1})

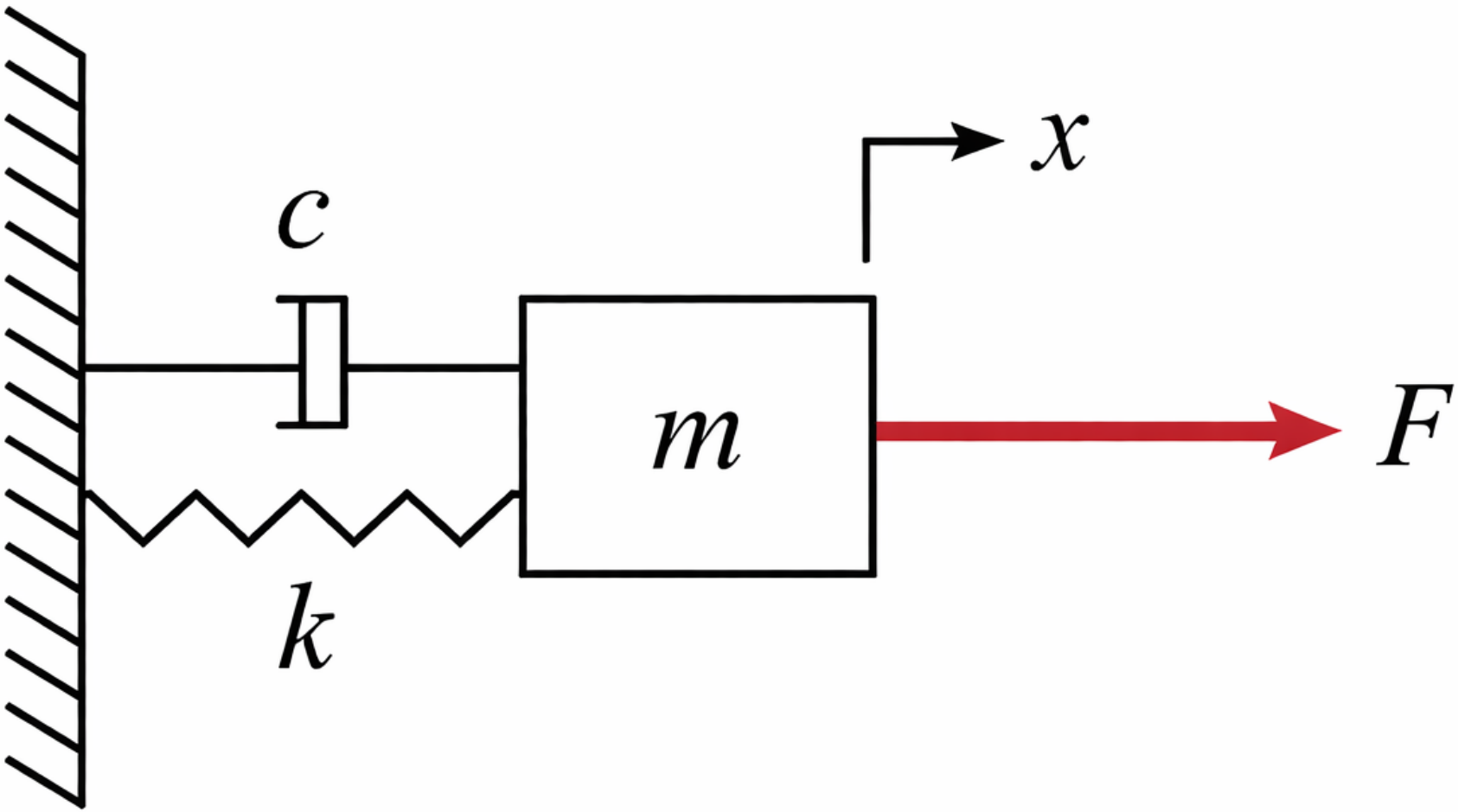
$$\bar{\boldsymbol{\mu}}_k = \boldsymbol{f}_d(\boldsymbol{\mu}_{k-1}, \boldsymbol{u}_{k-1}) \quad \text{(predicted state estimate)}$$
$$\bar{\Sigma}_k = F_k \Sigma_{k-1} F_k^\top + Q_{k-1} \quad \text{(predicted covariance estimate)}$$

2. **Correction Stage** (outputs: $\boldsymbol{\mu}_k$, Σ_k)

$$K_k = \bar{\Sigma}_k H_k^\top (H_k \bar{\Sigma}_k H_k^\top + R_k)^{-1} \quad \text{(near-optimal Kalman gain)}$$
$$\boldsymbol{\mu}_k = \bar{\boldsymbol{\mu}}_k + K_k \underbrace{(\boldsymbol{z}_k - \boldsymbol{h}_d(\bar{\boldsymbol{\mu}}_k))}_{\textit{innovation}} \quad \text{(corrected state estimate)}$$
$$\Sigma_k = (\mathcal{I} - K_k H_k)\bar{\Sigma}_k \quad \text{(corrected covariance estimate)}$$

[2] they substitute the matrices A_k and C_k of the linear case KF

Damped Harmonic Oscillator Exercise



Damped Harmonic Oscillator System

Damped Harmonic Oscillator Exercise - System Dynamics

The **Damped Harmonic Oscillator** is a second order **linear system**, that consists of a *double integrator* tied to a *spring*, which provides a restoring force toward a stable equilibrium (point or limit cycle), and a *damper*, which dissipates energy and reduces the motion over time. We assume:

- parameters: m (mass), c (damping coefficient), k (spring constant)
- state: $\boldsymbol{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \in \mathbb{R}^2$ contains the position and velocity of mass m
- control: $u = F \in \mathbb{R}$ is the scalar driving force of the system

We measure only the position, so the continuous-time state space model is:

$$\ddot{x} + \frac{c}{m}\dot{x} + \frac{k}{m}x = \frac{u}{m}$$
$$y = x$$

$$\Longrightarrow$$

$$\boxed{\begin{aligned} \dot{\boldsymbol{x}} &= \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u \\ y &= \begin{bmatrix} 1 & 0 \end{bmatrix} \boldsymbol{x} \end{aligned}}$$

Since this is a **linear system**, it can be **solved analytically**.

Damped Harmonic Oscillator Exercise - Discretized System

Since we have a linear system, its **discretization can be done analytically**, using for example **zero-order hold**. The *continuous-time state space model* of the system is:

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D = 0$$

So, the *discrete-time state space model* is analytically derived to be:

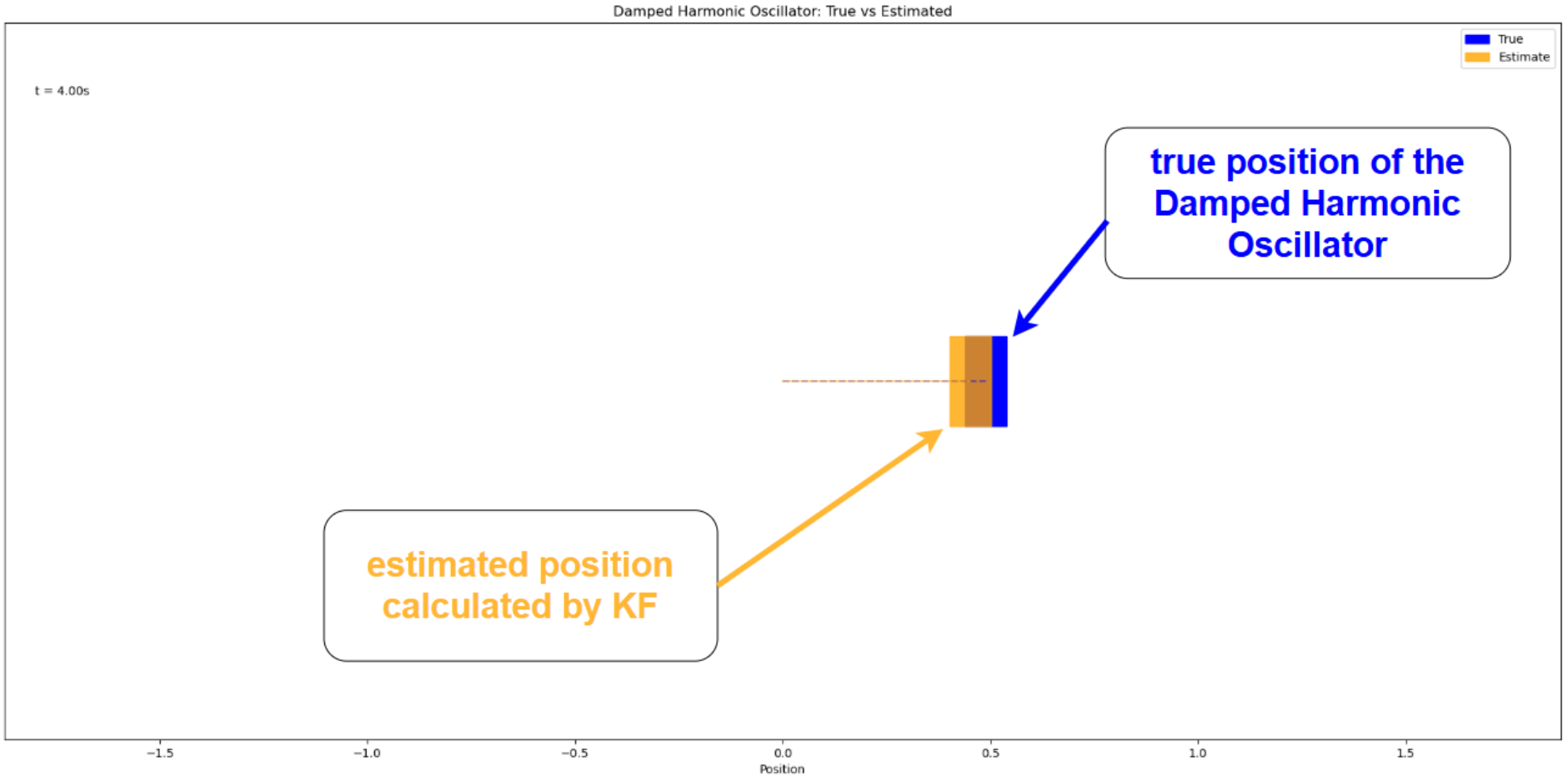
$$A_d = \frac{m}{\sqrt{c^2-4km}} \begin{bmatrix} \lambda_2 e^{\lambda_1 dt} - \lambda_1 e^{\lambda_2 dt} & e^{\lambda_2 dt} - e^{\lambda_1 dt} \\ \lambda_1\lambda_2(e^{\lambda_1 dt} - e^{\lambda_2 dt}) & \lambda_2 e^{\lambda_2 dt} - \lambda_1 e^{\lambda_1 dt} \end{bmatrix}, \quad \lambda_{1,2} = \frac{-c \mp \sqrt{c^2-4km}}{2m} \in \mathbb{C}$$

$$B_d = A^{-1}(A_d - \mathcal{I})B, \quad A^{-1} = \frac{m}{k} \cdot \begin{bmatrix} -\frac{c}{m} & -1 \\ \frac{k}{m} & 0 \end{bmatrix}, \qquad C_d = C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

We could also use the more easily derived *linear approximations*:

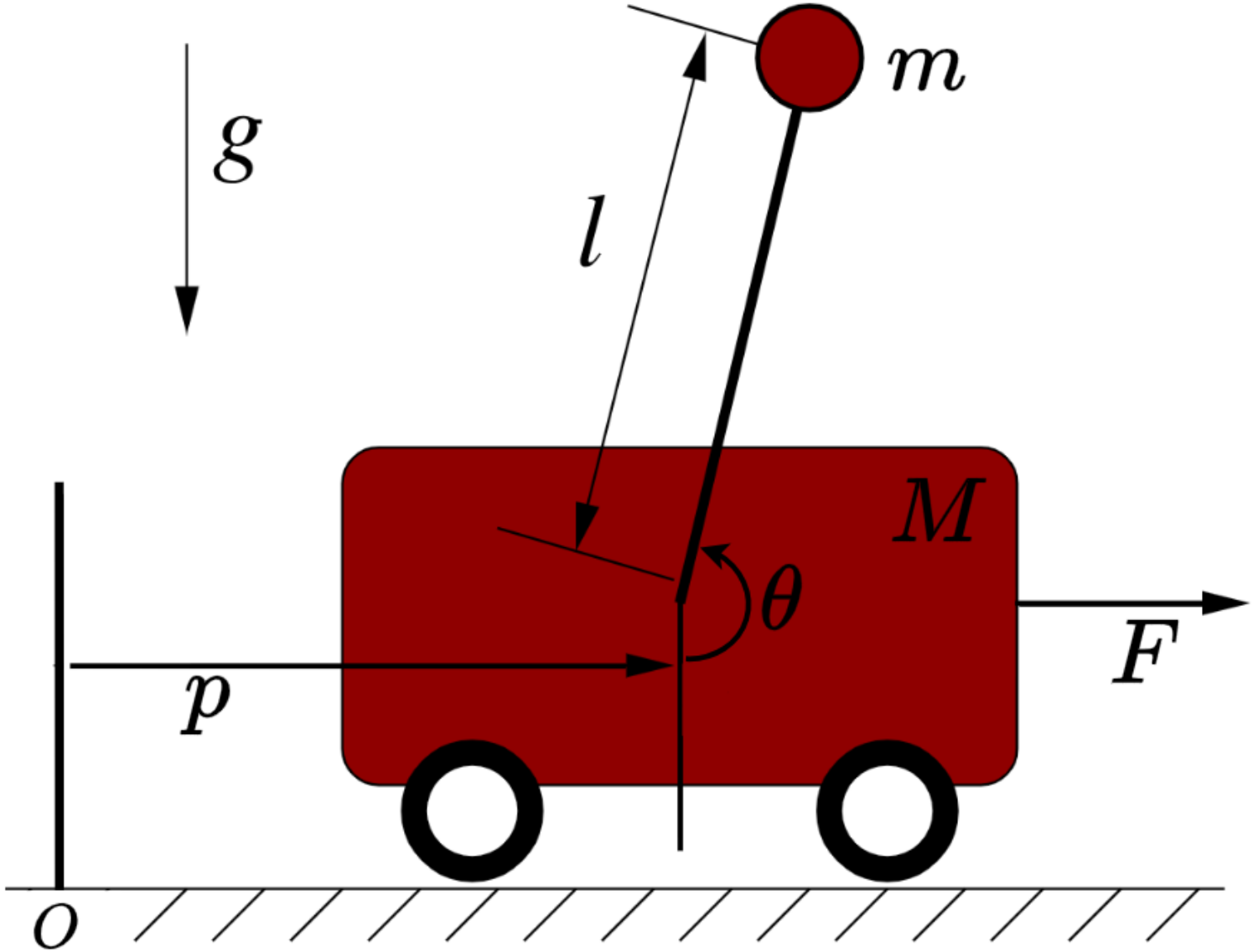
$$A_d = e^{Adt} \approx \mathcal{I} + Adt, \quad B_d = A^{-1}(e^{Adt} - \mathcal{I})B \approx Bdt$$

Damped Harmonic Oscillator Exercise - Example



Damped Harmonic Oscillator animation: true position vs KF estimated position

Cart-Pole Exercise



Cart-Pole System

Cart-Pole Exercise - System Dynamics

The **Cart-Pole** system consists of a *translating cart* (a *double integrator* under applied force) coupled to an *inverted pendulum*. This system introduces **nonlinear dynamics** and an unstable equilibrium. We assume:

- parameters: m (pendulum mass), M (cart mass), l (pole length)
- state: $\boldsymbol{x} = \begin{bmatrix} p & \dot{p} & \theta & \dot{\theta} \end{bmatrix}^\top \in \mathbb{R}^4$ contains the position and velocity of the cart, and the angular position and velocity of the pole
- control: $u = F \in \mathbb{R}$ is the scalar driving force of the system

We can derive its continuous-time state space model using Lagrangian mechanics (we assume that only the two position variables are observed):

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{p} \\ \ddot{p} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{p} \\ \frac{m \sin\theta (l\dot{\theta}^2 + g\cos\theta) + u}{M + m\sin^2\theta} \\ \dot{\theta} \\ \frac{-ml\dot{\theta}^2 \sin\theta\cos\theta - (m+M)g\sin\theta - u\cos\theta}{l(M + m\sin^2\theta)} \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \boldsymbol{x}$$

Cart-Pole Exercise - Discretized System & Jacobians

We already have showed that in order to derive the *discrete-time state space* matrices $F = A_d$ and B_d, ($H = C_d = C$ is constant), we need to first compute the Jacobians $A = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}$ and $B = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}$ of the *continuous-time system*. Here, we get:

$$A = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial \boldsymbol{f}}{\partial p} & \frac{\partial \boldsymbol{f}}{\partial \dot{p}} & \frac{\partial \boldsymbol{f}}{\partial \theta} & \frac{\partial \boldsymbol{f}}{\partial \dot{\theta}} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{\partial f_2}{\partial \theta} & \frac{\partial f_2}{\partial \dot{\theta}} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{\partial f_4}{\partial \theta} & \frac{\partial f_4}{\partial \dot{\theta}} \end{bmatrix}, \quad B = \frac{\partial \boldsymbol{f}}{\partial u} = \begin{bmatrix} 0 \\ \frac{1}{M + m\sin^2\theta} \\ 0 \\ \frac{-\cos\theta}{l(M + m\sin^2\theta)} \end{bmatrix}$$

Where:

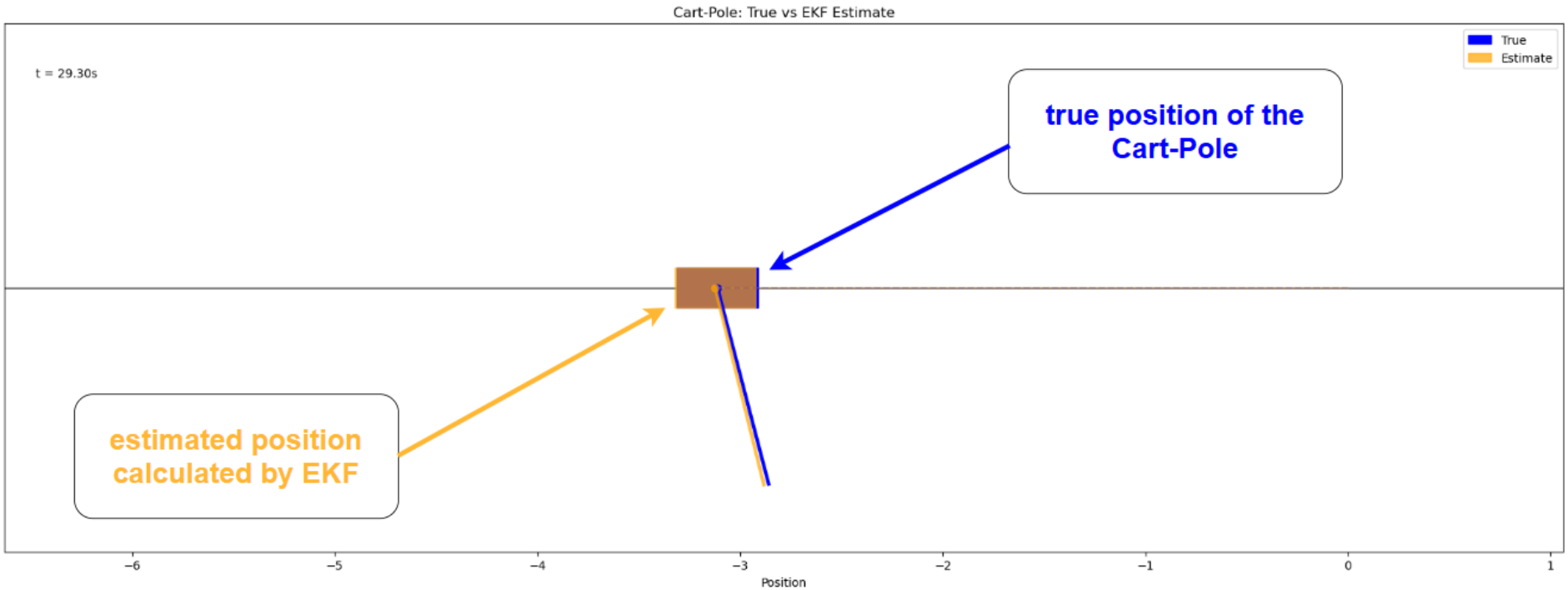
$$\frac{\partial f_2}{\partial \theta} = \frac{m(M - m\sin^2\theta)(g + l\dot{\theta}^2\cos\theta) - 2mMg\sin^2\theta - m\sin(2\theta)u}{(M + m\sin^2\theta)^2}$$

$$\frac{\partial f_2}{\partial \dot{\theta}} = \frac{2ml\dot{\theta}\sin\theta}{M + m\sin^2\theta}$$

$$\frac{\partial f_4}{\partial \theta} = \frac{-(ml\dot{\theta}^2 + g(m + M)\cos\theta)(M - m\sin^2\theta) + 2mMl\dot{\theta}^2\sin^2\theta + \sin\theta(m + m\cos^2\theta + M)u}{l(M + m\sin^2\theta)^2}$$

$$\frac{\partial f_4}{\partial \dot{\theta}} = -\frac{m\dot{\theta}\sin(2\theta)}{M + m\sin^2\theta}$$

Cart-Pole Exercise - Example



Cart-Pole animation: true position vs EKF estimated position

References

- State Estimation (wikipedia's article, control point of view)
- Discretization (wikipedia's article)
- Kalman Filter (wikipedia's article)
- Extended Kalman Filter (wikipedia's article)
- The Physics of the Damped Harmonic Oscillator
- The Physics of Cart-Pole
- State Estimation for Robotics; Timothy Barfoot (chapters 2, 3)
- Probabilistic Robotics; Sebastian Thrun, Wolfram Burgard, Dieter Fox (sections 2.1-2.5, 3.1, 3.2, 5.1-5.4, 6.3)
- Modern Robotics: Mechanics, Planning, and Control; Kevin M. Lynch and Frank C. Park